

25CS1302E - DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT

NAME: Poojitha Musunuri

ID: 2520030541

SECTION: 2B

LAB COURSEWORK

Experiment Title

Bank Transaction Management Using MongoDB Atlas Data Explorer

Problem Statement

A bank needs to manage information about its customers and their transactions.

The system should:

1. Store customer details in a **customers** collection.
2. Store transaction details in a **transactions** collection.
3. Use **customer_id** as the common field connecting customers and transactions.
4. Perform filtering operations on customer and transaction data.
5. Calculate total and average transaction amounts.
6. Group transactions by customer and transaction type.
7. Sort customer and transaction data.
8. Perform JOIN-like operations using **\$lookup**.
9. Generate complex transaction reports using MongoDB aggregation pipelines.

The assignment uses **customer_id** as the common field between the two collections.

Objective

To create a bank database using MongoDB Atlas, store customer and transaction information in separate collections, perform queries similar to SQL JOIN operations, and execute transaction analysis using MongoDB filters and aggregation pipelines.

Platform Used

- MongoDB Atlas
- MongoDB Atlas Data Explorer

Database Used

`bankdb`

Collections Used

1. `customers`
2. `transactions`

PROCEDURE:

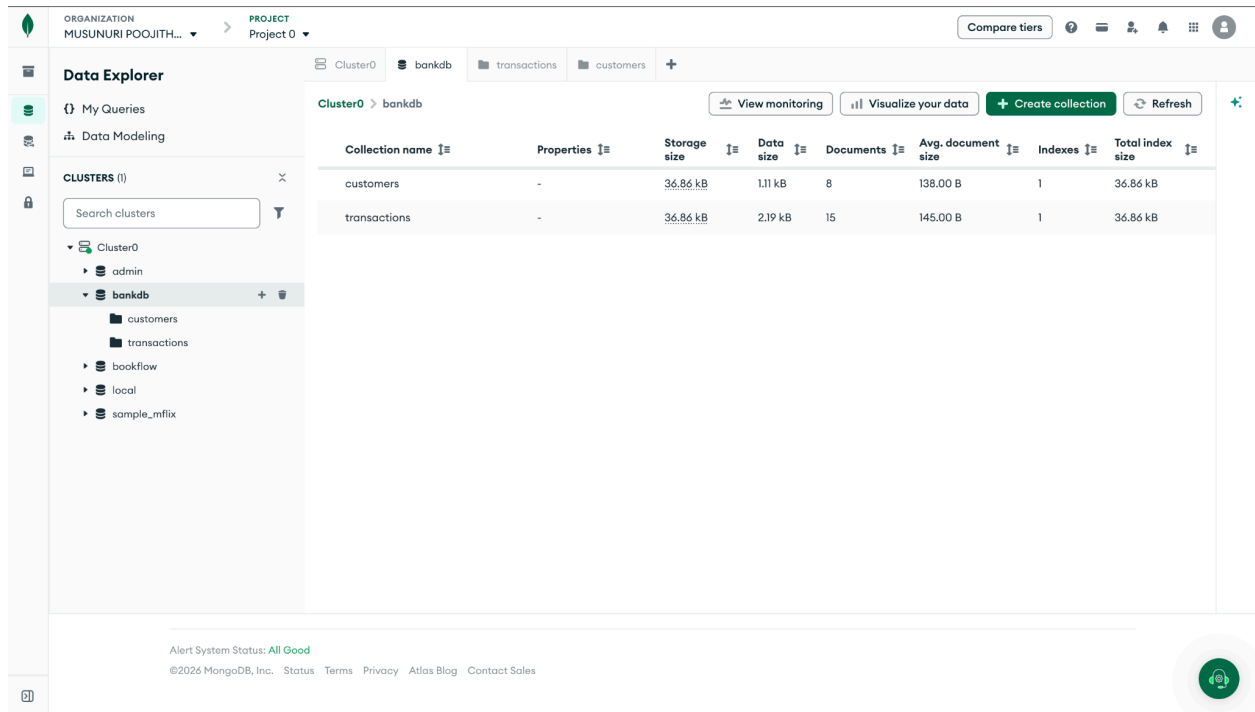
Step 1: Creating the Database and Collections

A MongoDB database named **bankdb** was created.

Two collections were created:

- `customers`
- `transactions`

Screenshot 1: Database and Collections Created



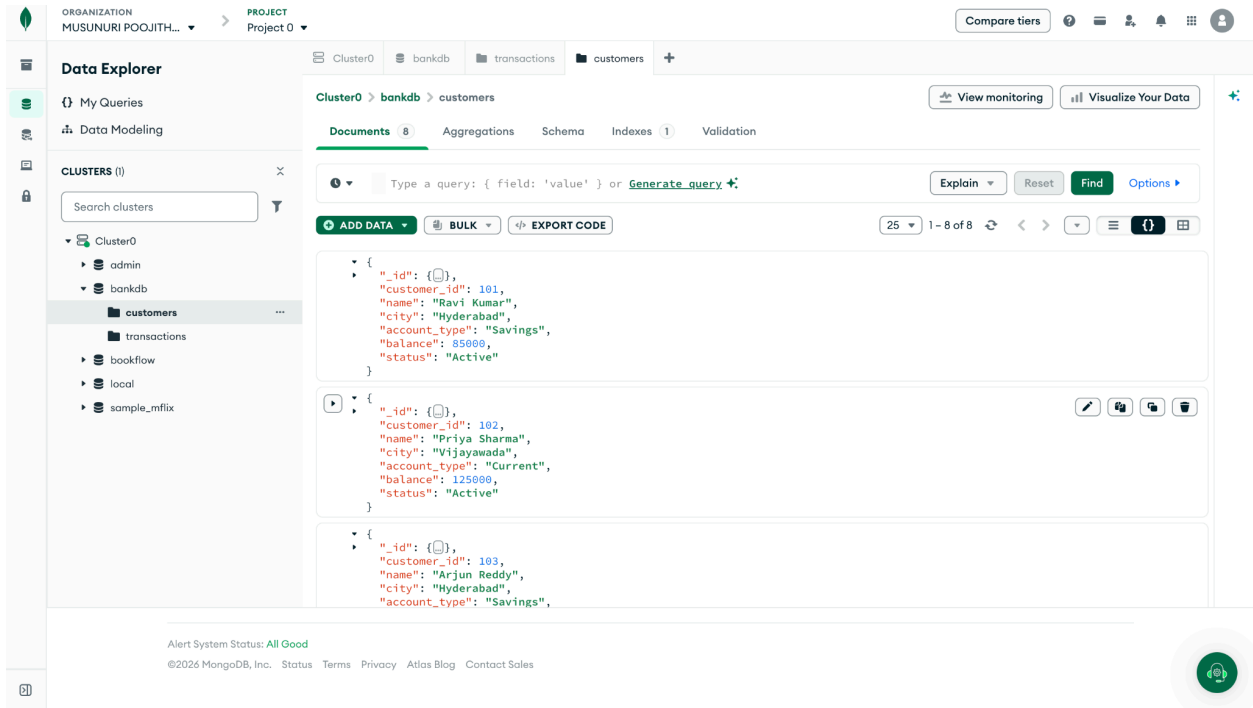
Step 2: Viewing Customer Documents

The documents stored in the `customers` collection were displayed.

Each customer document contains the following fields:

- `customer_id`
- `name`
- `city`
- `account_type`
- `balance`
- `status`

Screenshot 2: Customer Collection Documents



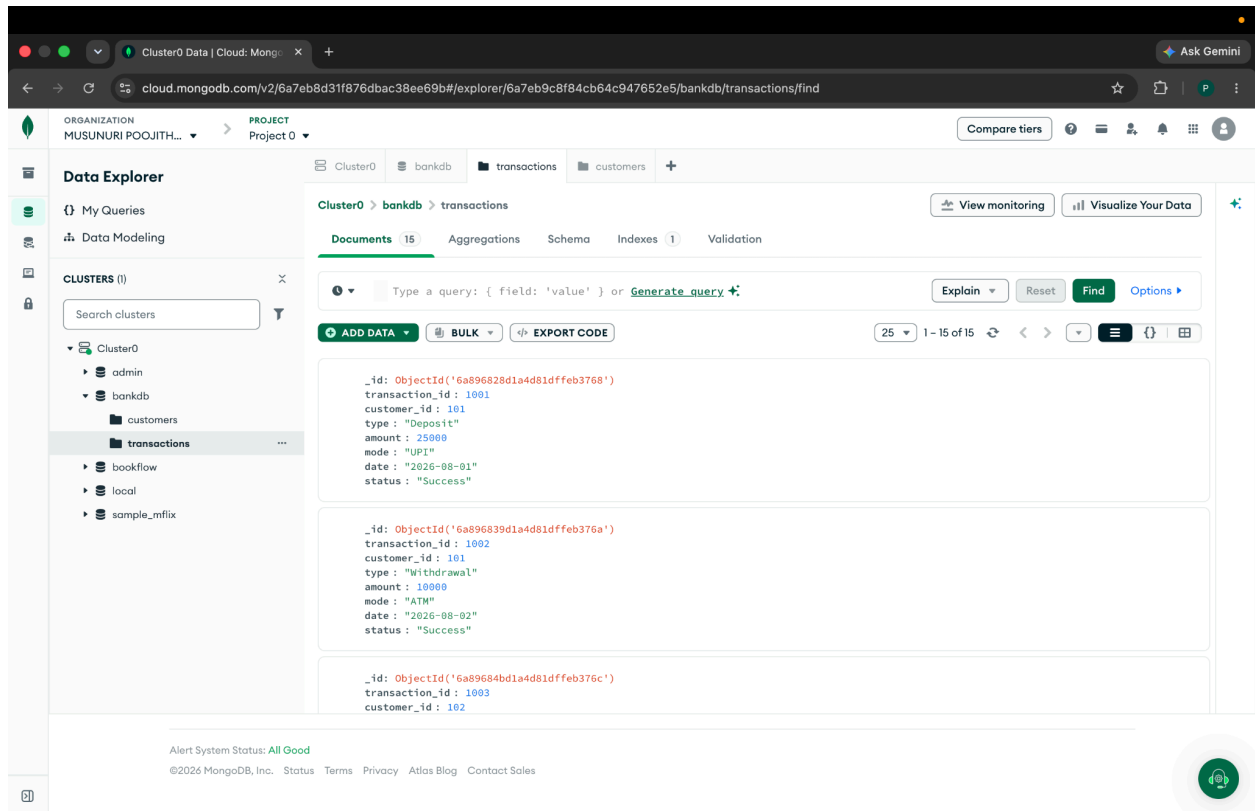
Step 3: Viewing Transaction Documents

The documents stored in the `transactions` collection were displayed.

Each transaction document contains information such as:

- `transaction_id`
- `customer_id`
- `type`
- `amount`
- `mode`
- `date`
- `status`

Screenshot 3: Transaction Collection Documents



Step 4: Query to Find Active Customers

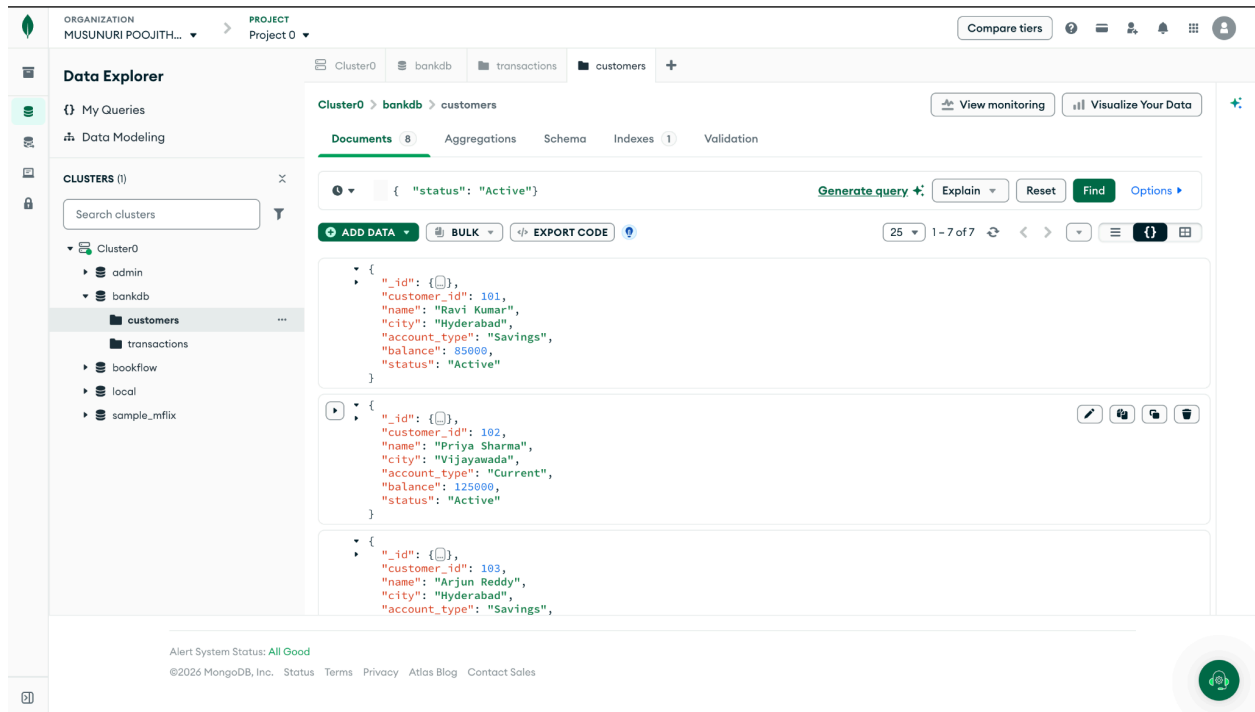
A query was performed to retrieve customers whose status is **Active**.

Query Used

```
{ "status": "Active" }
```

The query returned all customer documents having an active status.

Screenshot 4: Query Result for Active Customers



Step 5: Query to Find Customers with Balance Greater Than 100000

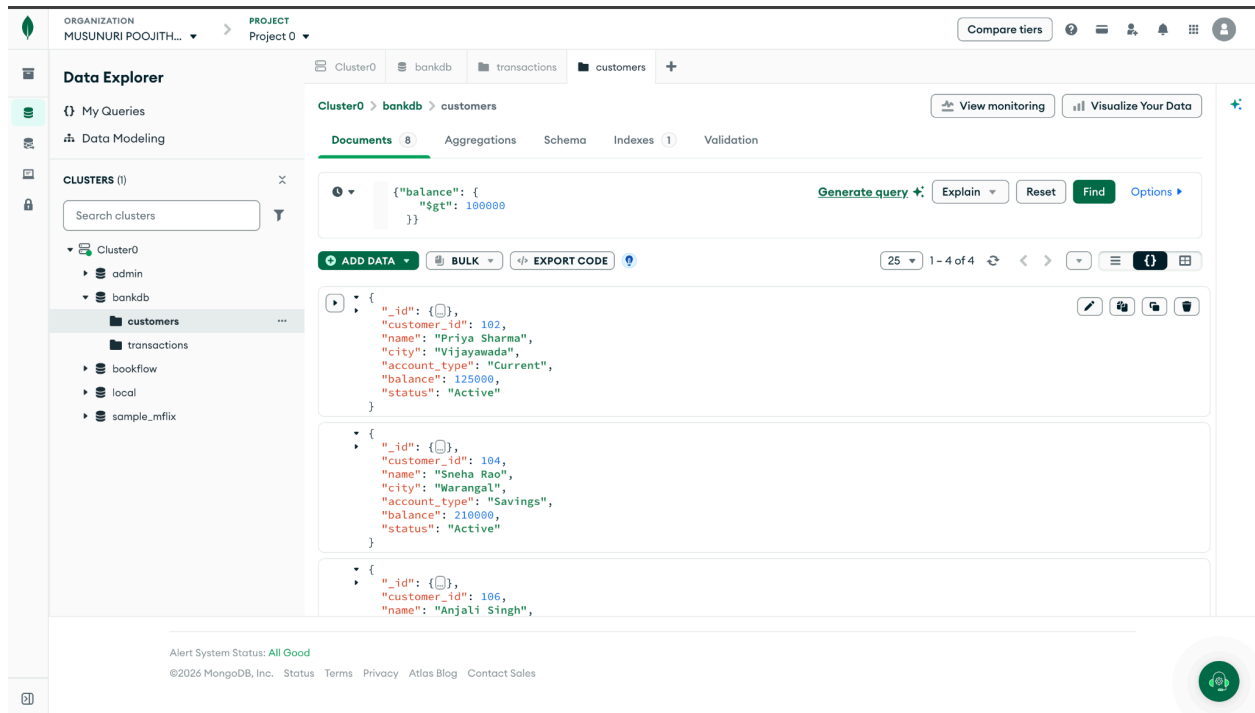
A query was performed to find customers whose account balance is greater than 100000.

Query Used

```
{  
  
  "balance": {  
  
    "$gt": 100000  
  
  }  
  
}
```

The `$gt` operator is used to retrieve documents where the value is greater than the specified value.

Screenshot 5: Customers with Balance Greater Than 100000



Step 6: Query to Find Customers from Hyderabad

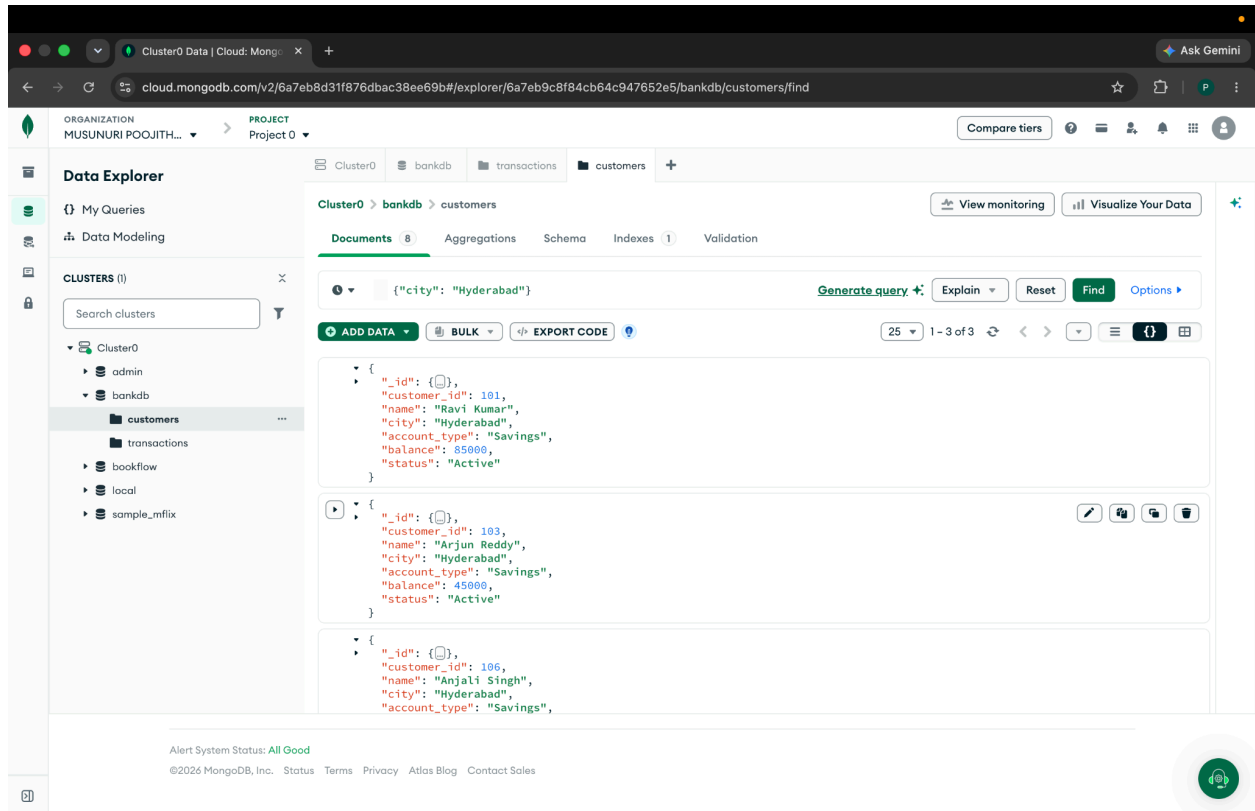
A query was performed to retrieve customers whose city is Hyderabad.

Query Used

```
{  
  "city": "Hyderabad"  
}
```

This query returned all customers whose **city** field contains the value **Hyderabad**.

Screenshot 6: Customers from Hyderabad



Step 7: Query to Find Savings Account Customers

A query was performed to retrieve customers whose account type is **Savings**.

Query Used

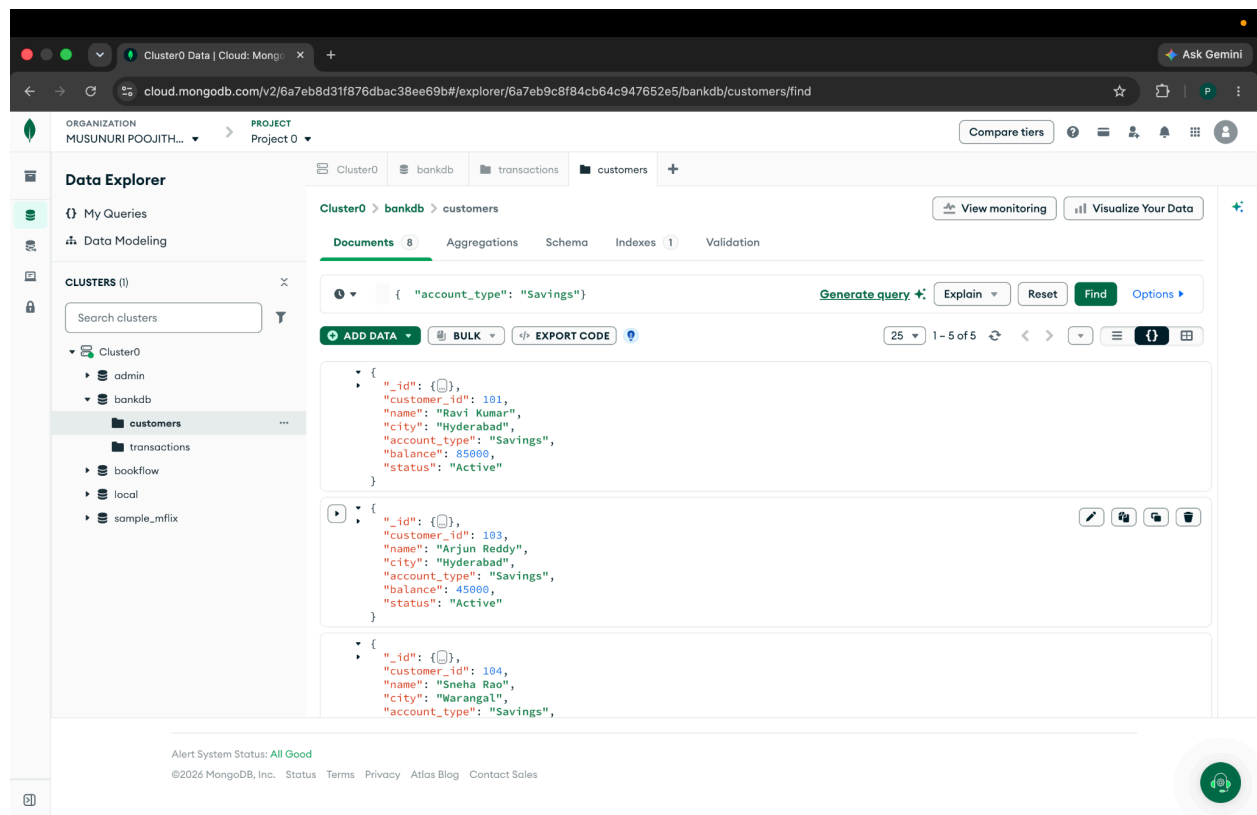
```
{
  "account_type": "Savings"
}
```

This query returned all customer documents where the **account_type** field contains the value **Savings**.

The customers retrieved were:

- Ravi Kumar
- Arjun Reddy
- Sneha Rao
- Anjali Singh
- Meena Devi

Screenshot 7: Query Result for Savings Account Customers



Step 8: Query to Find Transactions Greater Than 20000

The `transactions` collection was selected and a query was performed to retrieve transactions whose amount is greater than **20000**.

Query Used

```
{
  "amount": {
    "$gt": 20000
  }
}
```

```
}  
}
```

The `$gt` operator is used to retrieve documents where the value is greater than the specified value.

The transactions retrieved were:

- 1001
- 1003
- 1005
- 1007
- 1008
- 1010
- 1012
- 1014

Screenshot 8: Query Result for Transactions Greater Than 20000

The screenshot displays the MongoDB Cloud Data Explorer interface. The left sidebar shows the 'Data Explorer' section with a tree view of clusters. The main panel shows a query result for the 'transactions' collection in the 'bankdb' database. The query is `{ "amount": { "$gt": 20000 } }`. The results are displayed as a list of documents, with the first two visible:

```
{  
  "_id": ObjectId('6a896828d1a4d81dffe3768'),  
  "transaction_id": 1001,  
  "customer_id": 101,  
  "type": "Deposit",  
  "amount": 25000,  
  "mode": "UPI",  
  "date": "2026-08-01",  
  "status": "Success"  
}
```

```
{  
  "_id": ObjectId('6a89684bd1a4d81dffe376c'),  
  "transaction_id": 1003,  
  "customer_id": 102,  
  "type": "Deposit",  
  "amount": 50000,  
  "mode": "NEFT",  
  "date": "2026-08-02",  
  "status": "Success"  
}
```

The interface also includes a 'Generate query' button, an 'Explain' dropdown, a 'Reset' button, and a 'Find' button. The bottom of the screen shows the 'Alert System Status: All Good' and the MongoDB logo.

Step 9: Query to Find Deposit Transactions

A query was performed to retrieve all transactions whose type is **Deposit**.

Query Used

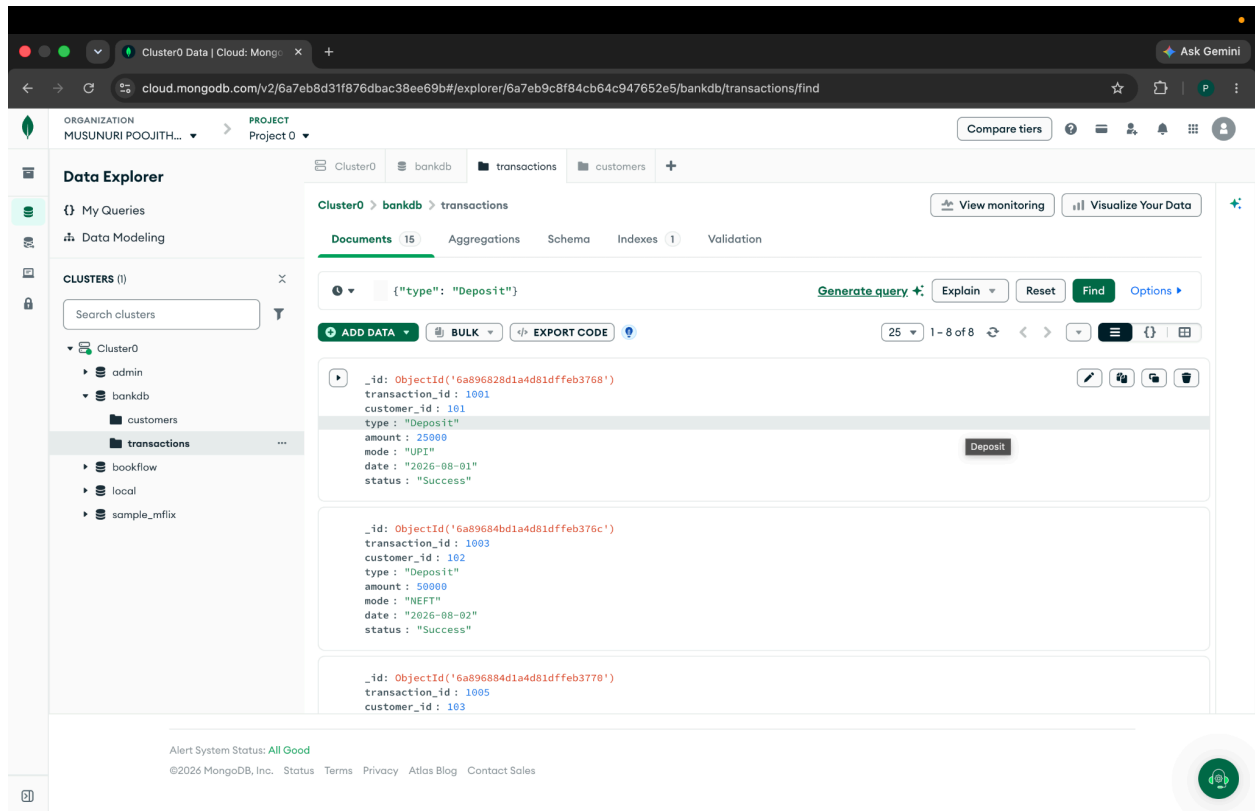
```
{  
  "type": "Deposit"  
}
```

This query returned all transaction documents where the **type** field contains the value **Deposit**.

The transactions retrieved were:

- 1001
- 1003
- 1005
- 1007
- 1009
- 1010
- 1012
- 1014

Screenshot 9: Query Result for Deposit Transactions



Step 10: Query to Find ATM Transactions

A query was performed to retrieve all transactions performed using the **ATM** mode.

Query Used

```
{
  "mode": "ATM"
}
```

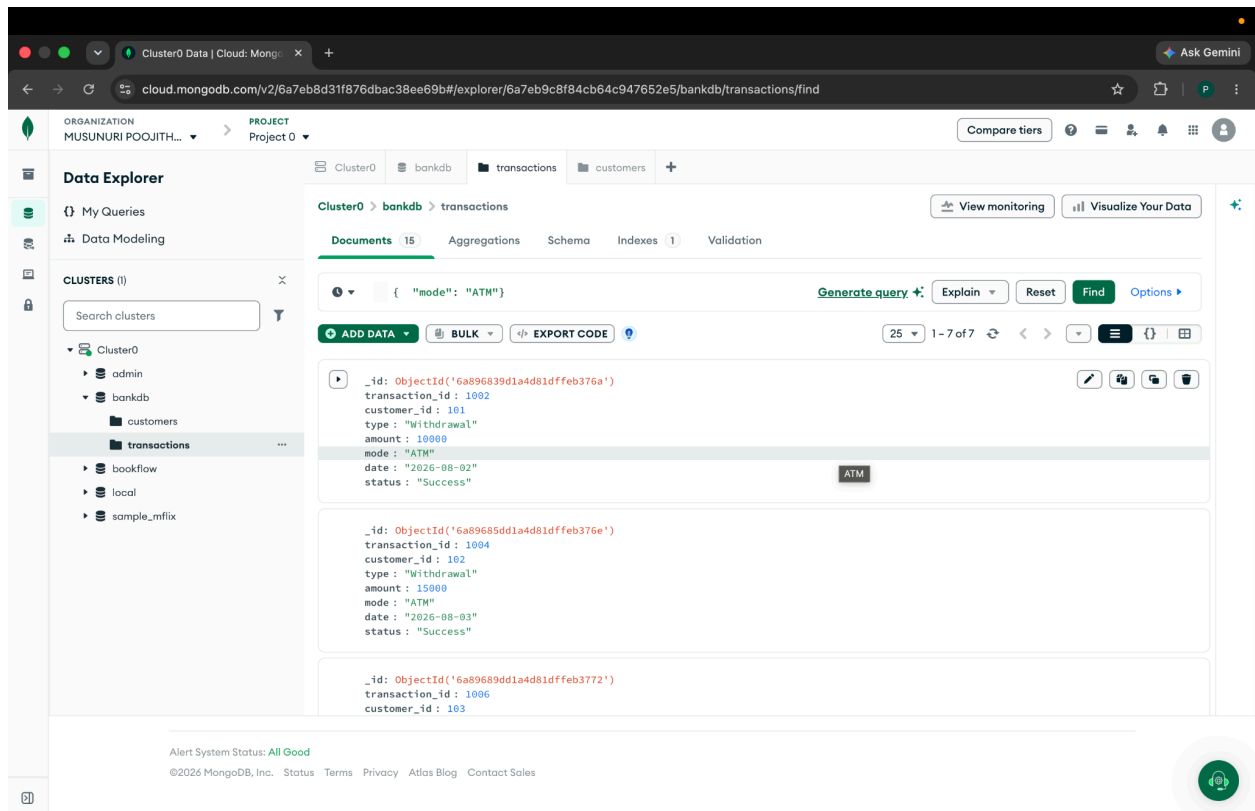
This query returned all transaction documents where the **mode** field contains the value **ATM**.

The transactions retrieved were:

- 1002
- 1004
- 1006

- 1008
- 1011
- 1013
- 1015

Screenshot 10: Query Result for ATM Transactions



Step 11: Query to Find Deposits Greater Than 50000

A query was performed to retrieve deposit transactions whose amount is greater than **50000**.

Query Used

```
{
  "type": "Deposit",
  "amount": {
    "$gt": 50000
  }
}
```

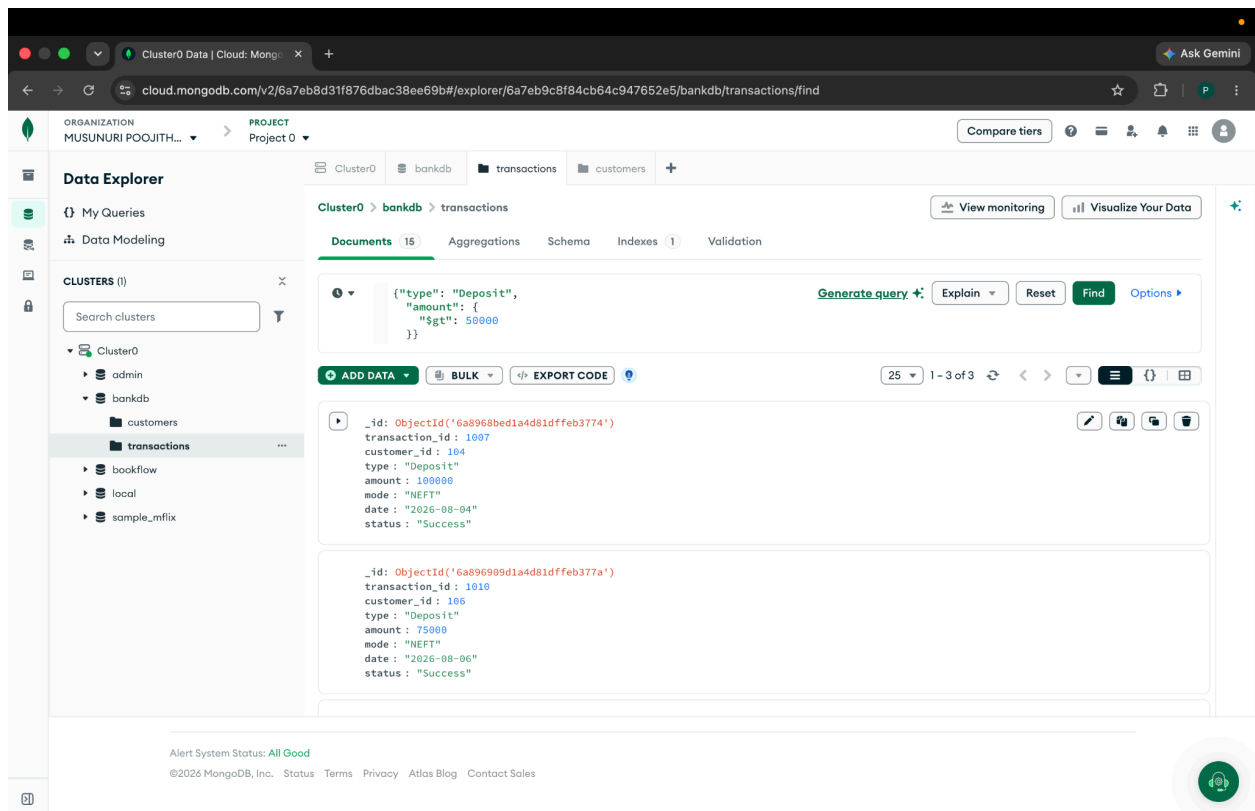
```
}  
}
```

This query uses multiple conditions. The transaction must have a type of **Deposit** and an amount greater than **50000**.

The transactions retrieved were:

- 1007
- 1010
- 1014

Screenshot 11: Query Result for Deposits Greater Than 50000



Step 12: Query to Find Hyderabad Customers with High Balance

A query was performed to retrieve customers from **Hyderabad** whose account balance is greater than **100000**.

Query Used

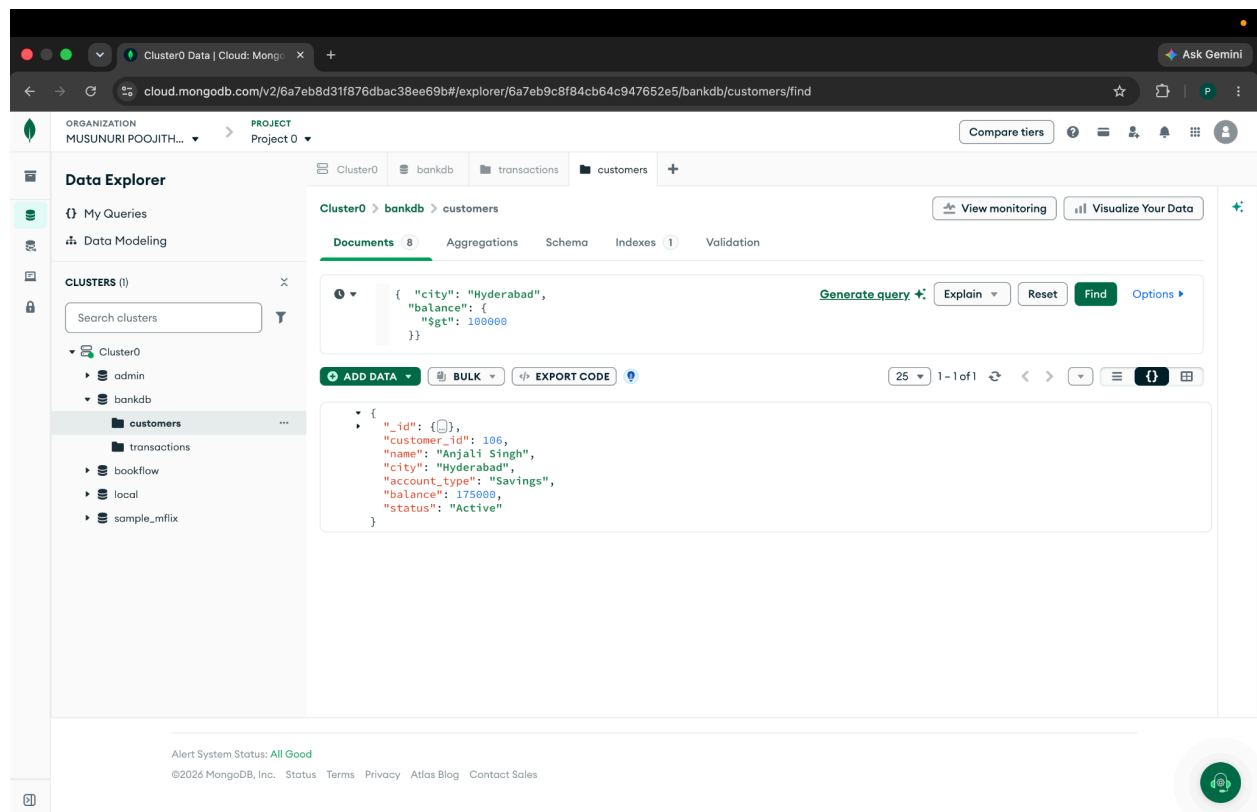
```
{
  "city": "Hyderabad",
  "balance": {
    "$gt": 100000
  }
}
```

The query uses two conditions: the customer must belong to Hyderabad and must have a balance greater than 100000.

The customer retrieved was:

- Anjali Singh

Screenshot 12: Query Result for Hyderabad Customers with High Balance



Step 13: Query to Find Customers in Multiple Cities

A query was performed using the `$in` operator to retrieve customers belonging to multiple specified cities.

Query Used

```
{
  "city": {
    "$in": [
      "Hyderabad",
      "Vijayawada",
      "Warangal"
    ]
  }
}
```

The `$in` operator is used to match documents where the field value is present in the specified list.

The customers retrieved were:

- Ravi Kumar
- Priya Sharma
- Arjun Reddy
- Sneha Rao
- Anjali Singh

Screenshot 13: Query Result for Customers in Multiple Cities

Cluster0 Data | Cloud: Mongo x + Ask Gemini

cloud.mongodb.com/v2/6a7eb8d31f876dbac38ee69b#/explorer/6a7eb9c8f84cb64c947652e5/bankdb/customers/find

ORGANIZATION MUSUNURI POOJITH... PROJECT Project 0

Cluster0 bankdb transactions customers +

Data Explorer

My Queries Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

admin bankdb customers transactions bookflow local sample_mflix

Cluster0 > bankdb > customers

Documents 8 Aggregations Schema Indexes 1 Validation

View monitoring Visualize Your Data

Generate query + Explain Reset Find Options

```
{
  "city": {
    "$in": [
      "Hyderabad",
      "Vijayawada",
      "Warangal"
    ]
  }
}
```

ADD DATA BULK EXPORT CODE

25 1 - 5 of 5

```
{
  "_id": {},
  "customer_id": 101,
  "name": "Ravi Kumar",
  "city": "Hyderabad",
  "account_type": "Savings",
  "balance": 85000,
  "status": "Active"
}
```

```
{
  "_id": {},
  "customer_id": 102,
  "name": "Priya Sharma",
  "city": "Vijayawada",
  "account_type": "Current",
  "balance": 125000,
  "status": "Active"
}
```

Alert System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Cluster0 Data | Cloud: Mongo x + Ask Gemini

cloud.mongodb.com/v2/6a7eb8d31f876dbac38ee69b#/explorer/6a7eb9c8f84cb64c947652e5/bankdb/customers/find

ORGANIZATION MUSUNURI POOJITH... PROJECT Project 0

Cluster0 bankdb transactions customers +

Data Explorer

My Queries Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

admin bankdb customers transactions bookflow local sample_mflix

Cluster0 > bankdb > customers

Documents 8 Aggregations Schema Indexes 1 Validation

View monitoring Visualize Your Data

Generate query + Explain Reset Find Options

```
{
  "city": {
    "$in": [
      "Hyderabad",
      "Vijayawada",
      "Warangal"
    ]
  }
}
```

ADD DATA BULK EXPORT CODE

25 1 - 5 of 5

```
{
  "_id": {},
  "customer_id": 103,
  "name": "Arjun Reddy",
  "city": "Hyderabad",
  "account_type": "Savings",
  "balance": 45000,
  "status": "Active"
}
```

```
{
  "_id": {},
  "customer_id": 104,
  "name": "Sneha Rao",
  "city": "Warangal",
  "account_type": "Savings",
  "balance": 210000,
  "status": "Active"
}
```

Alert System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Step 14: Sort Customers by Balance

The **Aggregations** option was used to sort customers according to their account balance in descending order.

The `$sort` stage was added.

Aggregation Used

```
{  
  "balance": -1  
}
```

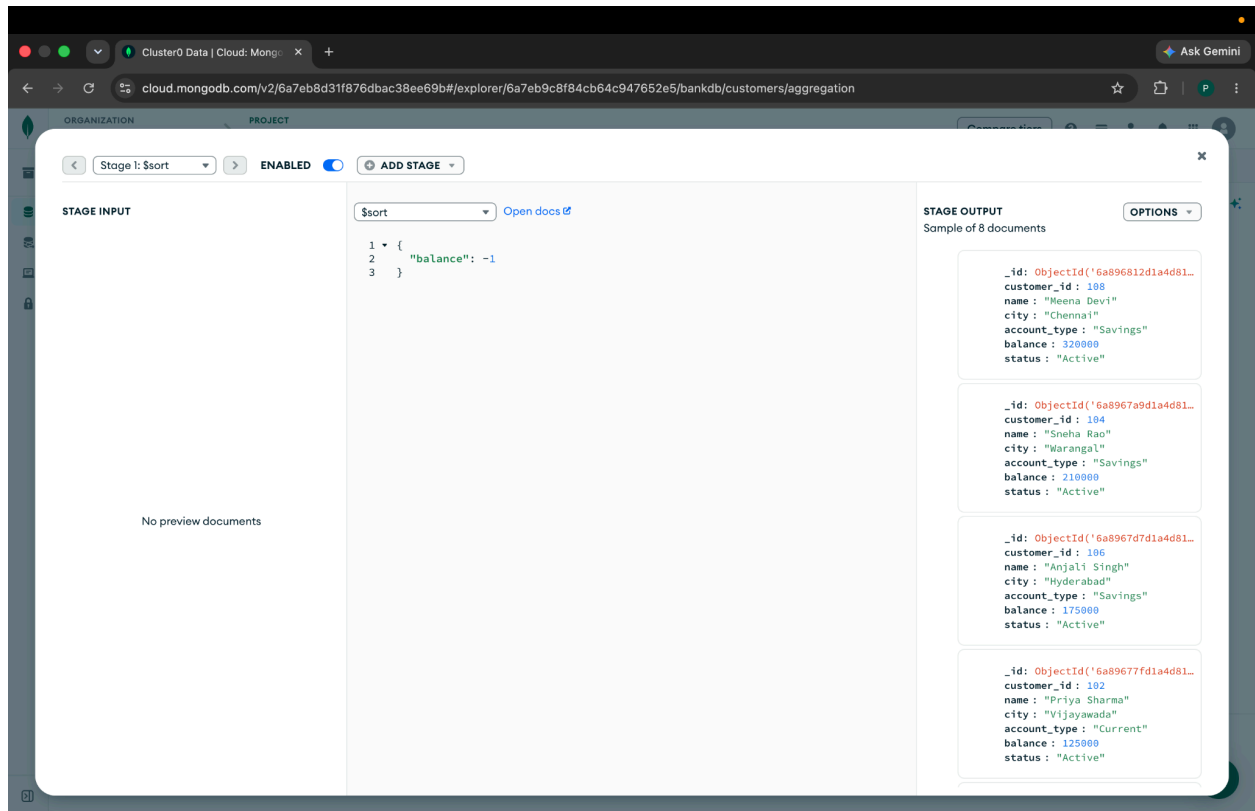
The value `-1` indicates descending order.

The customers were sorted from the highest balance to the lowest balance.

Expected order:

- Meena Devi – 320000
- Sneha Rao – 210000
- Anjali Singh – 175000
- Priya Sharma – 125000
- Rahul Verma – 95000
- Ravi Kumar – 85000
- Kiran Kumar – 65000
- Arjun Reddy – 45000

Screenshot 14: Customers Sorted by Balance



Step 15: Calculate Total Transaction Amount

The **transactions** collection was selected and the **\$group** aggregation stage was used to calculate the total transaction amount.

Aggregation Used

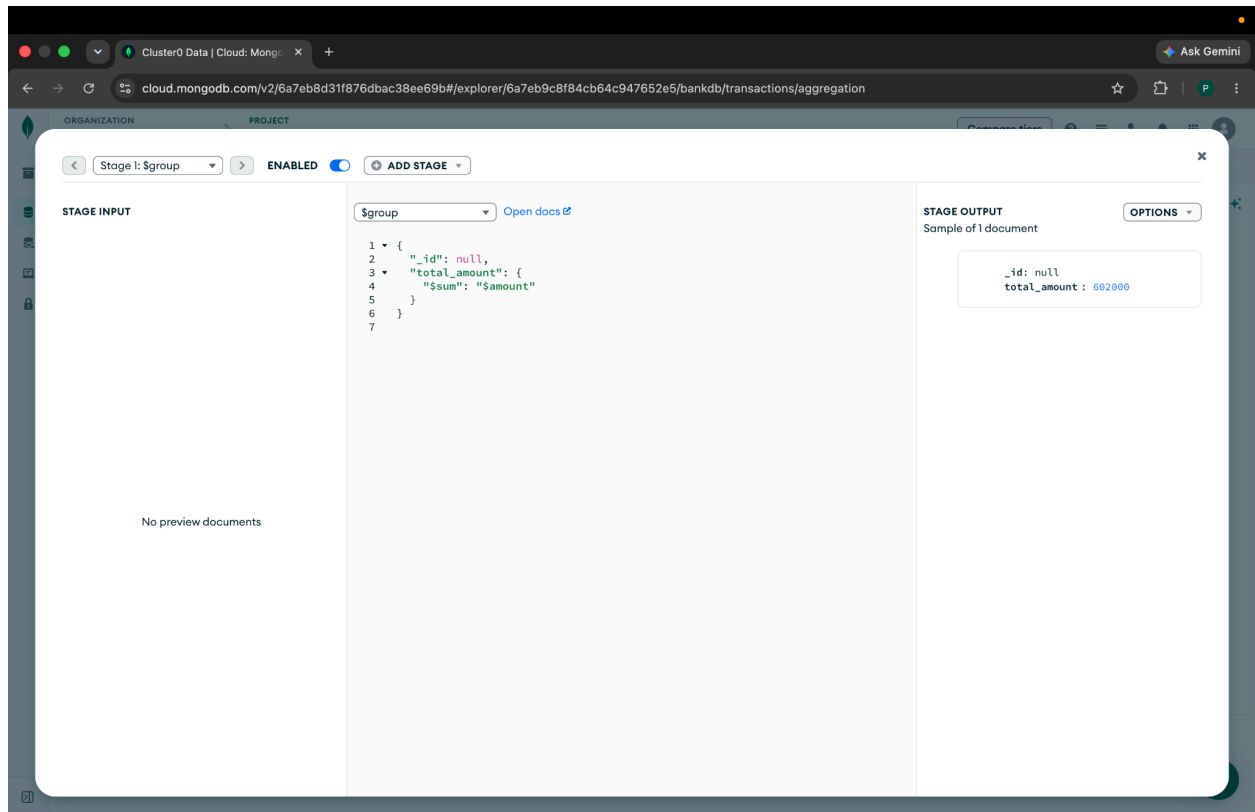
```
{
  "_id": null,
  "total_amount": {
    "$sum": "$amount"
  }
}
```

The **\$sum** operator was used to calculate the sum of all transaction amounts.

The result obtained was:

total_amount = 632000

Screenshot 15: Total Transaction Amount



Step 16: Calculate Average Transaction Amount

An aggregation query was performed to calculate the average amount of all transactions.

Aggregation Used

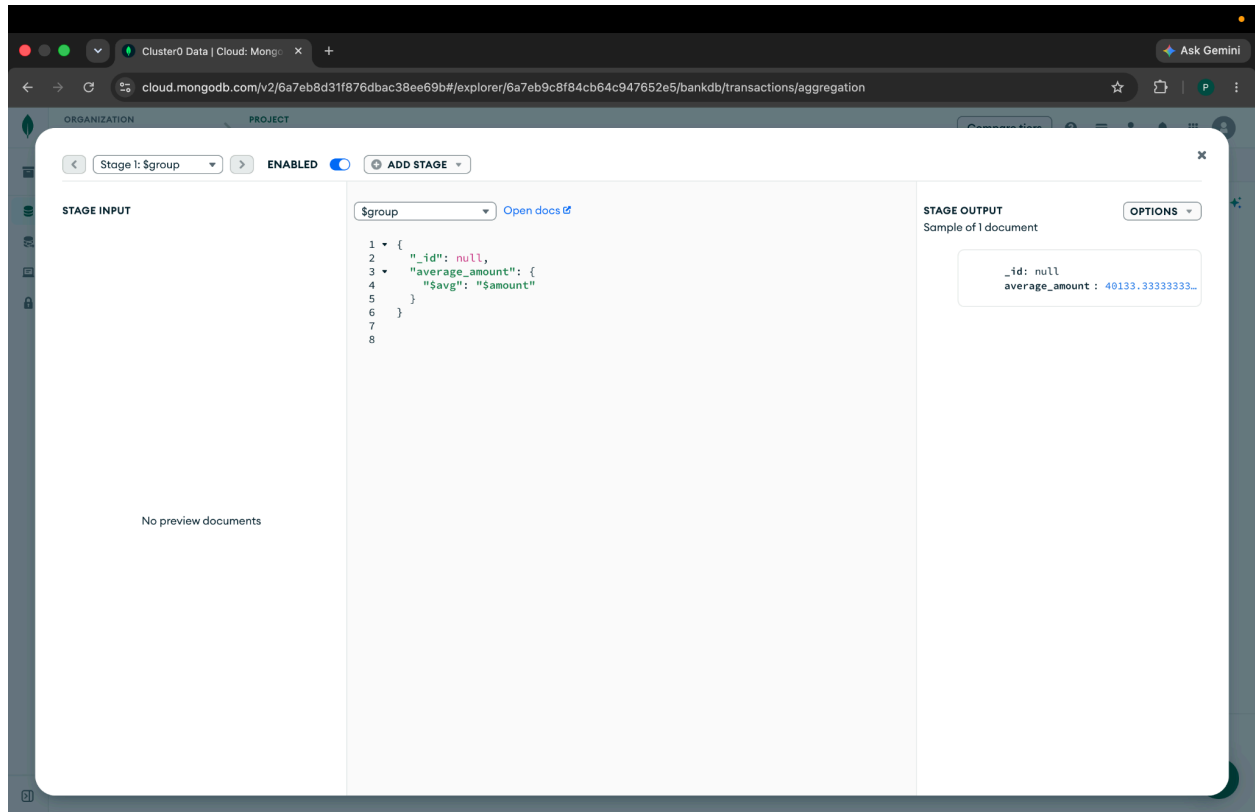
```
{
  "_id": null,
  "average_amount": {
    "$avg": "$amount"
  }
}
```

The **\$avg** operator was used to calculate the average value of the **amount** field.

The result obtained was:

average_amount = 42133.33

Screenshot 16: Average Transaction Amount



Step 17: Calculate Total Transaction Amount by Type

An aggregation query was performed to calculate the total transaction amount separately for each transaction type.

The `$group` stage was used to group transactions based on the `type` field.

Aggregation Used

```
[
  {
    "$group": {
```

```

    "_id": "$type",
    "total_amount": {
      "$sum": "$amount"
    }
  }
}
]

```

The **\$sum** operator was used to calculate the total amount for each transaction type.

The results obtained were:

Deposit: 485000

Withdrawal: 147000

Screenshot 17: Total Transaction Amount by Type

The screenshot displays the MongoDB Cloud Portal interface for an aggregation pipeline. The pipeline is named '\$group' and is applied to the 'transactions' collection in the 'bankdb' database. The pipeline consists of a single stage, 'Stage 1', which uses the '\$group' operator to calculate the total amount for each transaction type. The output preview shows two documents: one for 'Withdrawal' with a total amount of 117000, and one for 'Deposit' with a total amount of 485000.

Aggregation Pipeline:

```

1 {
2   "_id": "$type",
3   "total_amount": {
4     "$sum": "$amount"
5   }
6 }

```

Output preview after \$group stage (Sample of 2 documents):

_id	total_amount
Withdrawal	117000
Deposit	485000

Step 18: Calculate Number of Transactions per Customer

An aggregation query was performed to calculate the number of transactions made by each customer.

The `$group` stage was used to group transactions according to `customer_id`.

The `$sum` operator with the value `1` was used to count the transactions.

Aggregation Used

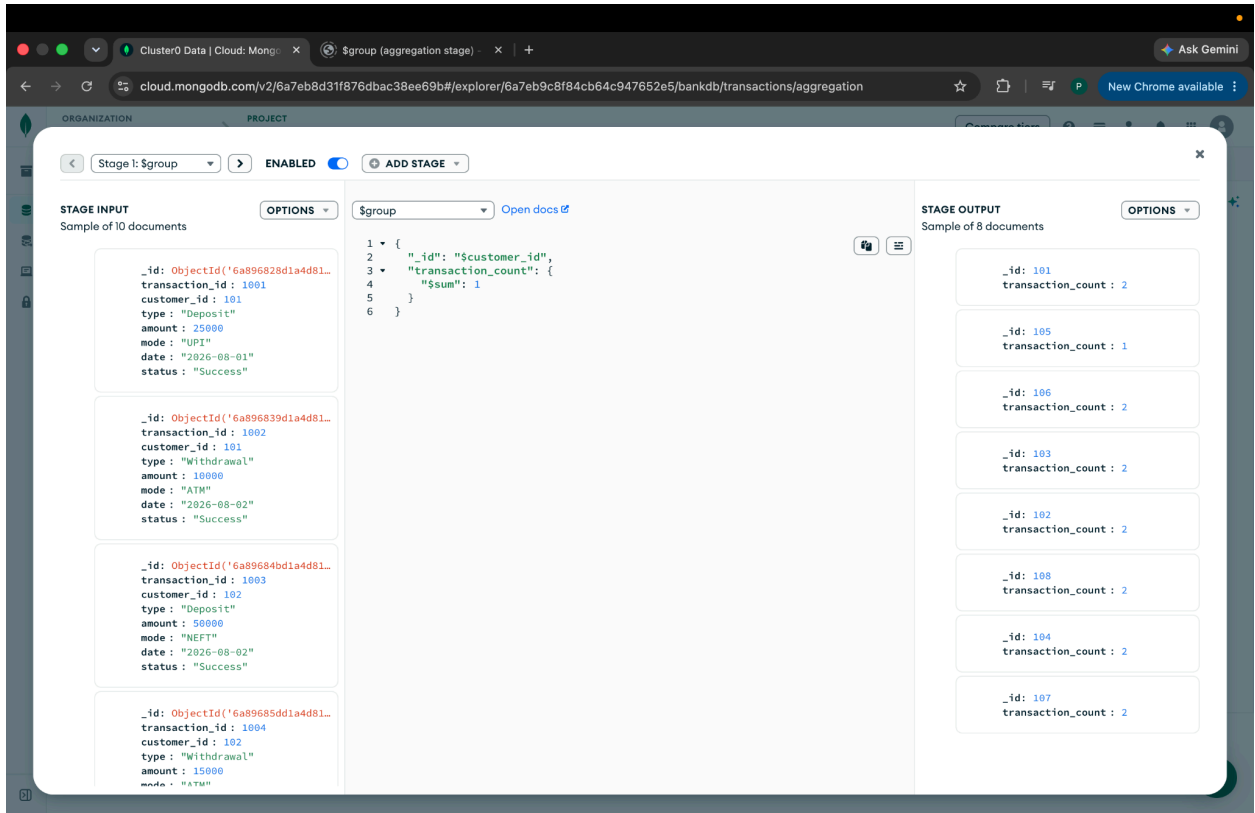
```
[
  {
    "$group": {
      "_id": "$customer_id",
      "transaction_count": {
        "$sum": 1
      }
    }
  },
  {
    "$sort": {
      "transaction_count": -1
    }
  }
]
```

The `$sort` stage was then used to display the customers according to the number of transactions in descending order.

The expected results were:

101	2
102	2
103	2
104	2
106	2
107	2
108	2
105	1

Screenshot 18: Number of Transactions per Customer



This screenshot shows the first stage of an aggregation pipeline in MongoDB Atlas. The stage is named "Stage 1: \$group" and is currently enabled. The input is a sample of 10 documents, and the output is a sample of 8 documents. The pipeline is configured to group documents by customer_id and calculate the transaction_count for each group.

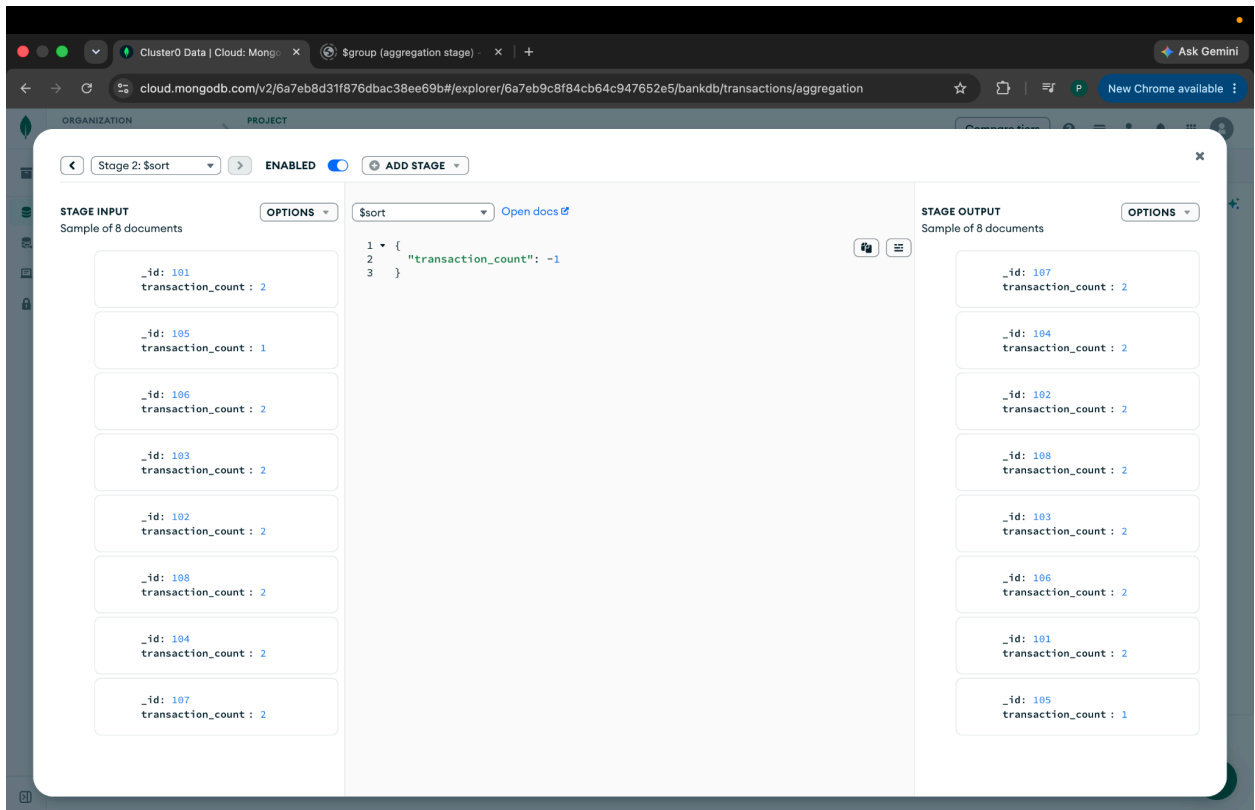
Stage 1: \$group (ENABLED)

STAGE INPUT (Sample of 10 documents)

```
1 {
2   "_id": "customer_id",
3   "transaction_count": {
4     "$sum": 1
5   }
6 }
```

STAGE OUTPUT (Sample of 8 documents)

_id	transaction_count
101	2
105	1
106	2
103	2
102	2
108	2
104	2
107	2



This screenshot shows the second stage of the aggregation pipeline in MongoDB Atlas. The stage is named "Stage 2: \$sort" and is currently enabled. The input is a sample of 8 documents, and the output is a sample of 8 documents. The pipeline is configured to sort the documents by transaction_count in descending order.

Stage 2: \$sort (ENABLED)

STAGE INPUT (Sample of 8 documents)

```
1 {
2   "transaction_count": -1
3 }
```

STAGE OUTPUT (Sample of 8 documents)

_id	transaction_count
107	2
104	2
102	2
108	2
103	2
106	2
101	2
105	1